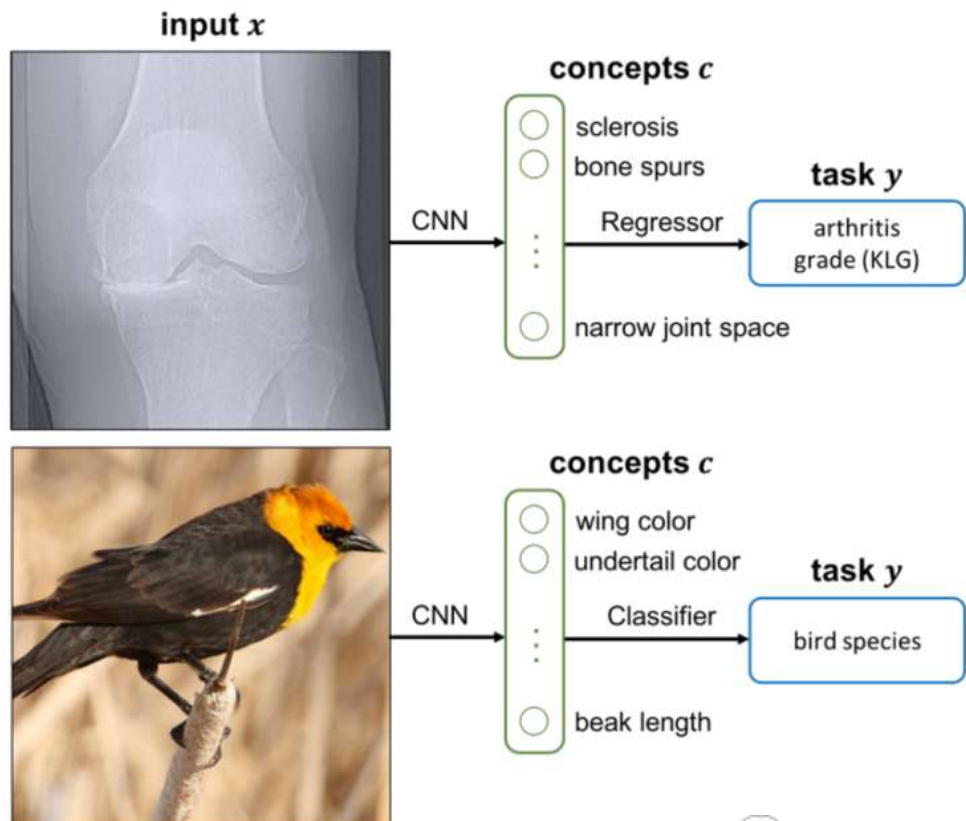


Concept Bottleneck Models

概念瓶颈模型

ICML 2020



模型定义与数学形式

设输入为 x ，目标为 y ，概念向量为 $c \in \mathbb{R}^k$ 。 (x, c, y)

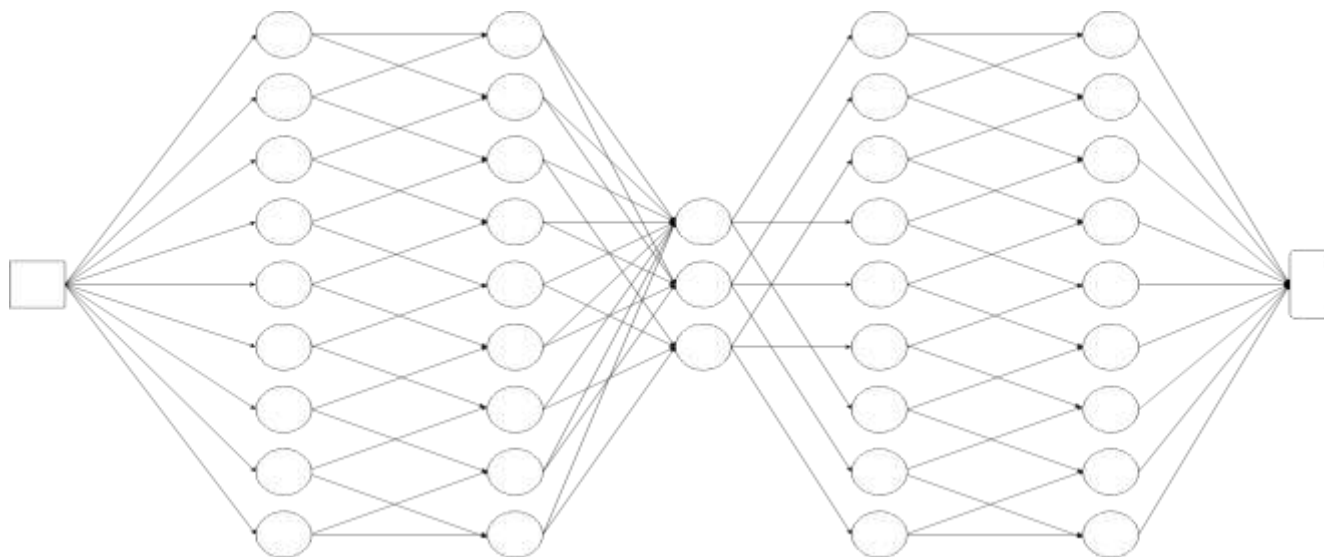
CBM 由两个映射组成：

- 概念预测器 $g: x \rightarrow \hat{c}$

- 目标预测器 $f: \hat{c} \rightarrow \hat{y}$

最终预测为：

$$\hat{y} = f(g(x))$$



独立瓶颈 (Independent Bottleneck)

流程:

- 单独训练 g : 使用输入 x 和概念标签 c , 最小化概念预测损失 $\mathcal{L}_c(g(x), c)$

对于连续概念常用均方误差 (MSE), 对于二值概念常用二元交叉熵 (BCE)。

- 单独训练 f : 使用**真实概念值** c 和目标标签 y , 最小化任务损失 $\mathcal{L}_y(f(c), y)$:

回归任务用 MSE, 分类任务用交叉熵。

特点:

- f 在训练时从未见过预测概念 $\hat{c}$ 的分布, 因此测试时输入 $\hat{c}$ 会面临分布偏移。
- 但如果测试时能获得真实概念 (如人工干预), 将 $\hat{c}$ 替换为 c 后, f 的表现最佳, 因为它正是在真实概念上训练的。

顺序瓶颈 (Sequential Bottleneck)

流程:

- 首先训练 g (同独立瓶颈)。
- 固定 g , 用其预测的 $\hat{c} = g(x)$ 和 y 训练 f , 最小化 $\mathcal{L}_y(f(\hat{c}), y)$ 。

特点:

- f 适应了预测概念的分布, 因此无干预时任务准确率通常高于独立瓶颈。
- 但干预时替换为真实概念后, f 可能无法充分利用真实概念, 因为它的训练分布是 $\hat{c}$ 而非 c 。

联合瓶颈 (Joint Bottleneck)

流程：

- 同时优化两个目标，使用加权和损失函数：

$$\mathcal{L} = \lambda \mathcal{L}_c(g(x), c) + \mathcal{L}_y(f(\hat{c}), y)$$

- 其中 $\lambda > 0$ 控制概念损失与任务损失的相对重要性。

特点：

- 通过调节 λ 可以在允许模型在保证任务精度的同时调整概念表示，使概念预测和任务预测相互适应。
- 概念准确率和任务准确率之间权衡 ($\lambda \rightarrow 0$ 退化为标准模型, $\lambda \rightarrow \infty$ 近似顺序瓶颈)。

MODEL	y RMSE (OAI)	y ERROR (CUB)
INDEPENDENT	0.435 ± 0.024	0.240 ± 0.012
SEQUENTIAL	0.418 ± 0.004	0.243 ± 0.006
JOINT	0.418 ± 0.004	0.199 ± 0.006
STANDARD	0.441 ± 0.006	0.175 ± 0.008
NO BOTTLENECK	0.443 ± 0.008	0.173 ± 0.003
MULTITASK	0.425 ± 0.010	0.162 ± 0.002

MODEL	y ERROR	c ERROR
STANDARD	0.627 ± 0.013	-
JOINT	0.482 ± 0.018	0.069 ± 0.002
SEQUENTIAL	0.496 ± 0.009	0.072 ± 0.002
INDEPENDENT	0.482 ± 0.008	0.072 ± 0.002

Flexible Concept Bottleneck Model

灵活概念瓶颈模型

AAAI, 2026

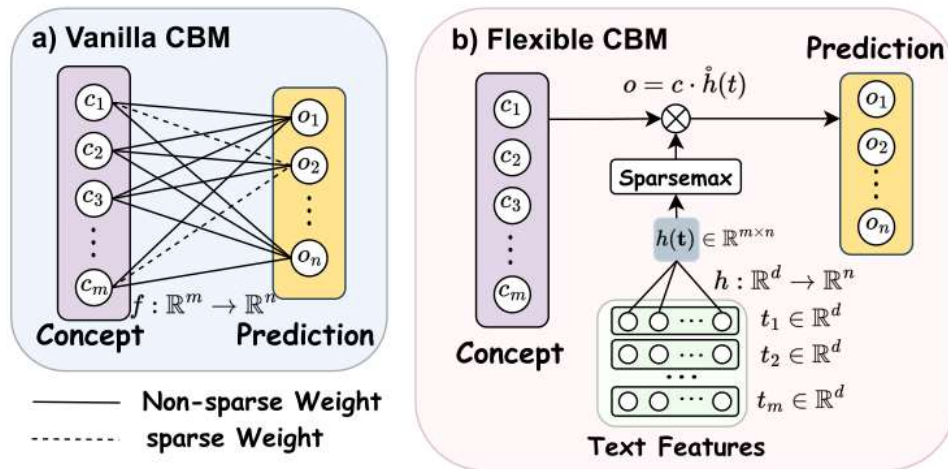
研究背景与动机

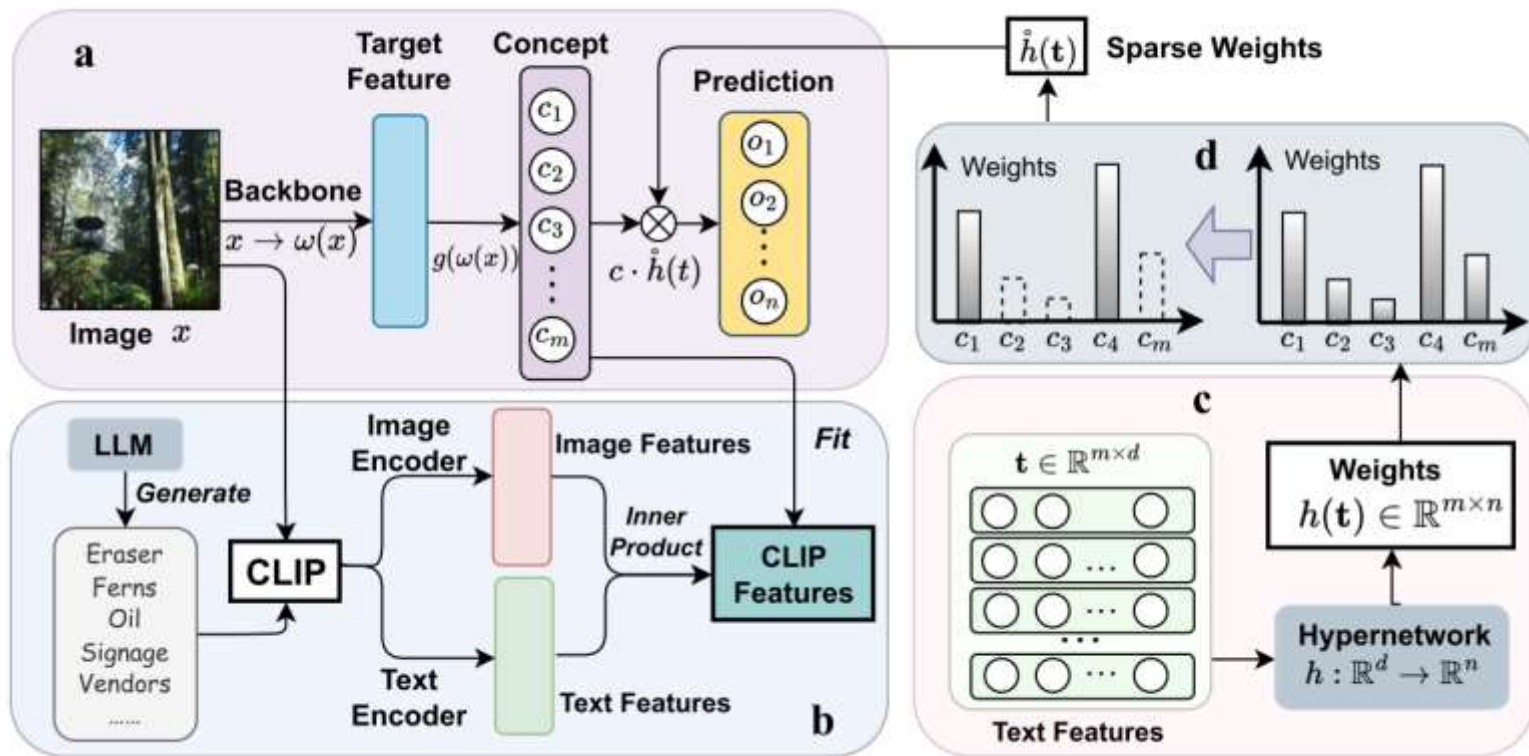
背景：

- 概念瓶颈模型**：传统CBMs通过引入一个人类可理解的概念中间层（如“翅膀颜色”、“栖息地”），使神经网络决策过程透明化。模型先预测概念，再基于概念做最终分类。
- VLM-based CBMs**：为了降低对昂贵人工标注概念的依赖，现有工作利用大语言模型生成概念词，并用视觉-语言模型（如CLIP）计算图像与概念的相似度。

动机：

现有的VLM-based CBMs有一个致命缺陷：**概念池是固定的**。一旦训练完成，如果要新增概念（如医学新发现）或更换底层VLM导致语义向量变化，就必须**重新训练整个模型**。这在VLM技术快速迭代（GPT-4o、DeepSeek-V3等层出不穷）的当下，严重限制了模型的可扩展性和部署灵活性。





第一阶段：离线概念生成与特征提取（数据准备）

1.1 LLM 概念生成

输入：数据集的类别名称列表

方法：调用大语言模型，并使用预设的提示词

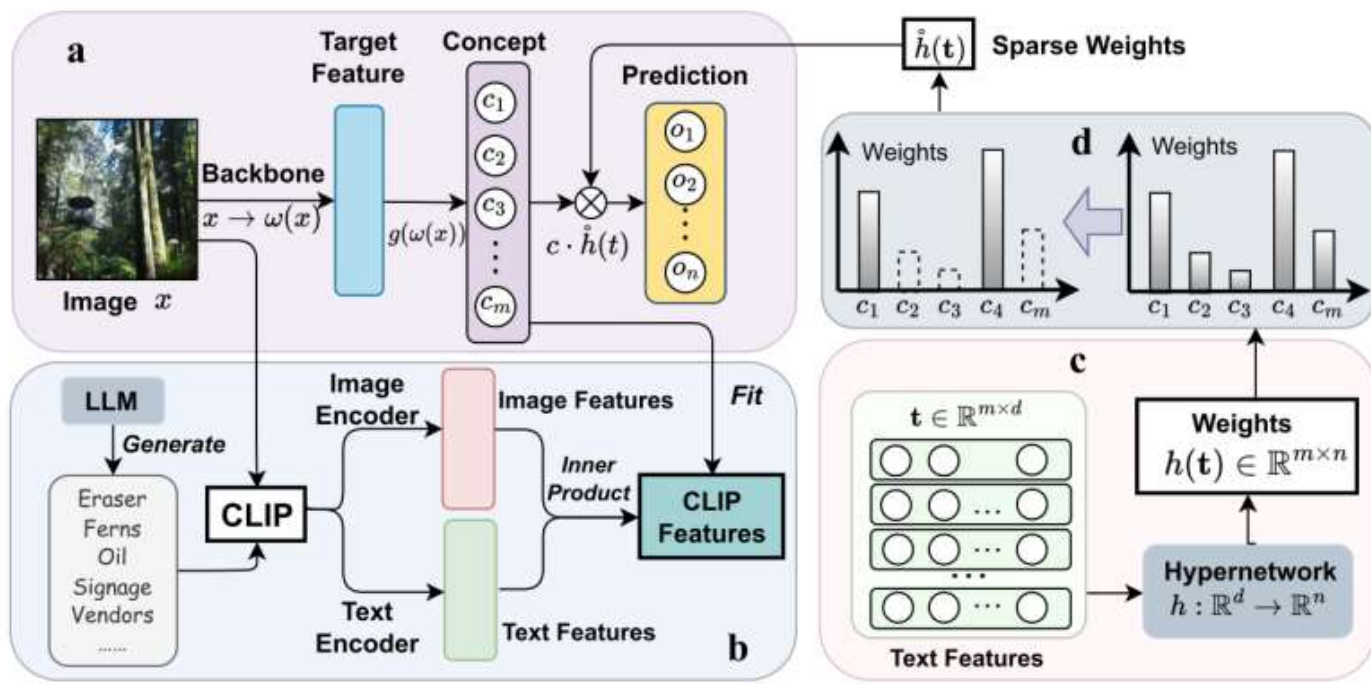
输出：每个类别获得一组文本描述概念词，汇总成全局概念池，共 m 个概念。

1.2 概念文本编码

输入：概念池中 m 个概念的文本描述

方法：使用预训练的 CLIP 文本编码器提取特征

输出：文本特征矩阵



第二阶段：训练阶段

子阶段 A：训练概念预测器 g

图像编码

输入：图像

方法：使用预训练的 CLIP 视觉编码器提取特征。

输出：图像特征向量

概念相似度计算

输入：图像特征和文本特征

方法：相乘，计算余弦相似度

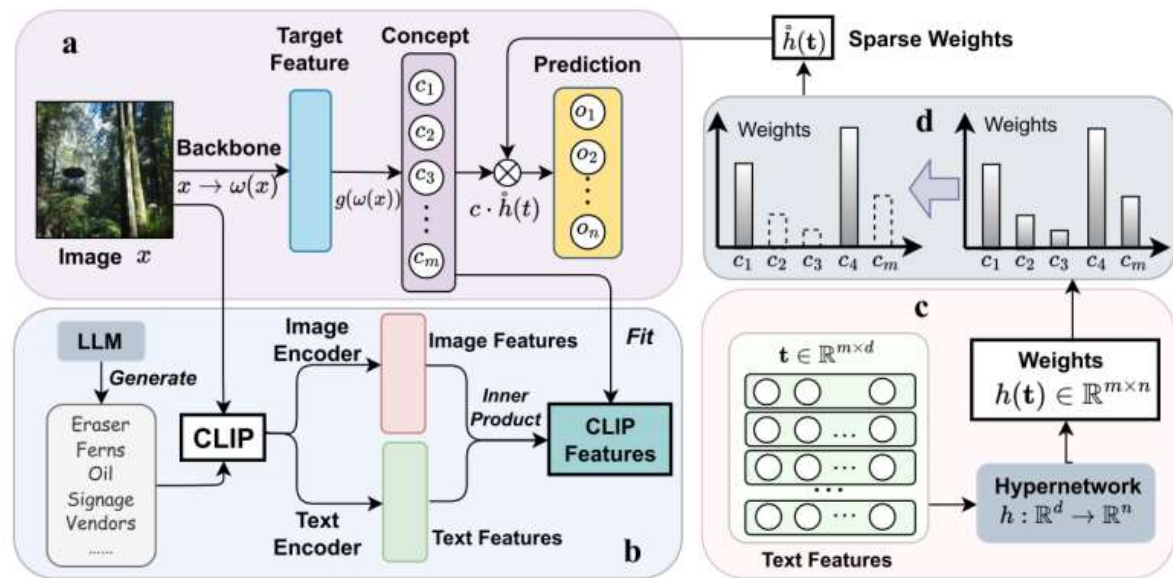
输出：概念激活矩阵

损失优化

输入：图像

方法：将图像输入骨干网络，得出特征，再经过投影头 g 得出概念值，再与概念激活矩阵作损失。

优化目标：投影头 g



子阶段B：训练超网络 h 与稀疏化模块

目标：学习如何根据概念文本动态生成分类权重，并强制该权重稀疏以保持可解释性。

超网络前向传播

目标：将固定的文本特征转化为分类权重。

输入：文本特征矩阵

方法：输入到超网络 h 进行处理

输出：稠密权重矩阵 $h(t)$

分类决策

目的：结合视觉概念激活与语义权重进行最终预测。

输入：图像概念向量、稀疏权重

方法：直接相乘，再进行softmax

输出：类别概率

稀疏化投影

目标：剔除无关概念的权重，仅保留 Top-K 关键概念。

输入：稠密权重 $h(t)$ 和可学习温度 τ

方法：应用 Sparsemax 算法

输出：稀疏权重矩阵

损失反向传播

目的：更新超网络参数和温度 τ

【准备阶段】 (VLMDInterface.preprocess)

1. LLM 生成概念集合 → 存为 concept_set txt 文件
2. 用 CLIP 对概念文本编码 → text_features [num_concepts, 768]
save_clip_text_features()
3. 用 backbone (如 clip_ViT-L/14) 对训练/验证图像编码
→ target_features [N, 768] (图像的视觉特征, 用于概念投影)
→ clip_features_image [N, 768] (CLIP 图像特征)
4. clip_features = clip_features_image @ text_features.T
→ [N, num_concepts] (每张图对每个概念的相关度分数)
5. 按 clip_cutoff 过滤不活跃概念, 得最终 num_concepts

【阶段一训练】 (FCBMConceptInterface)

输入批次: (target_features, clip_features, labels)

target_features [N, 768]

↓ VLMConceptFCBM (单线性层)

outs_c [N, num_concepts]

损失: loss_c = -cos_similarity_cubed_single(clip_features, outs_c)
(最小化损失 = 最大化投影输出与 CLIP 图文相关度的对齐)

测试时: 保存全量训练集和验证集的 outs_c 到磁盘 (.pt 文件)

【阶段二训练】 (FCBMHyperInterface)

输入: 加载阶段一保存的 outs_c, 归一化 (标准化)

输入批次: (outs_c [N, num_concepts], labels)

+ text_features [num_concepts, 768]

text_features → ThreeLayerMLP → weights [num_concepts, num_class]

↓ sparsemax (稀疏化)

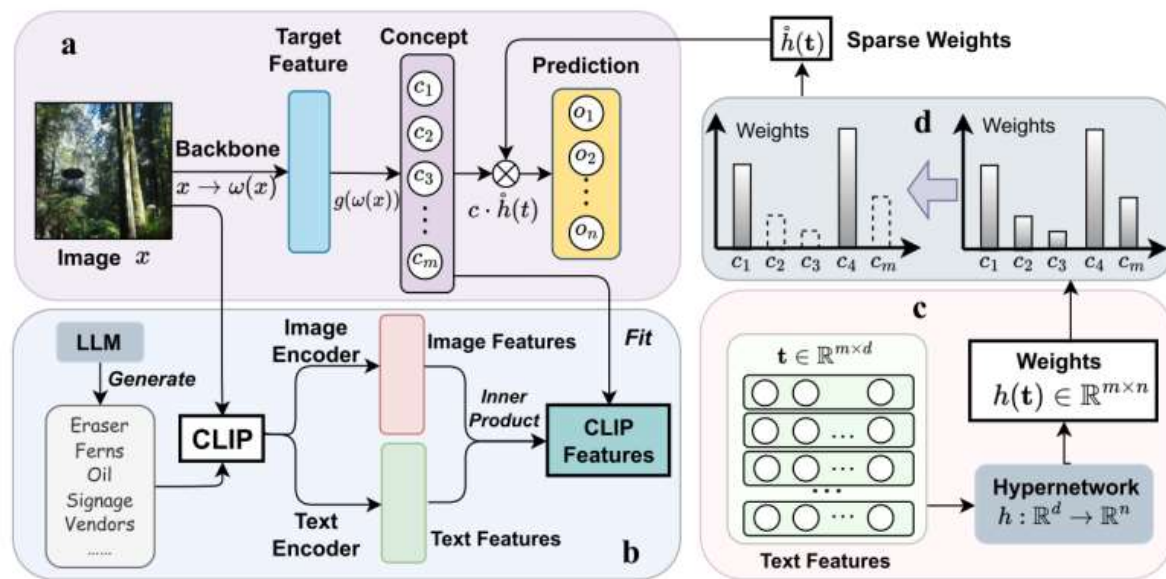
outs_c [N, num_concepts] @ weights [num_concepts, num_class]

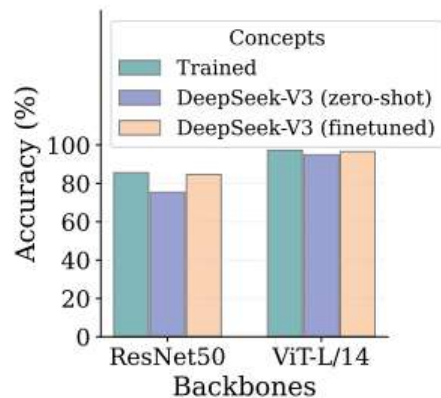
↓

outs_y [N, num_class]

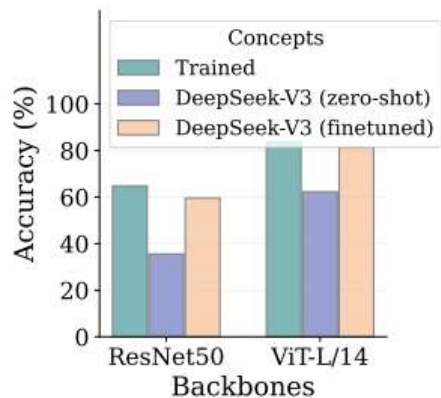
损失: CrossEntropyLoss(outs_y, labels)

ThreeLayerMLP: 768 → 4096 → 4096 → num_class

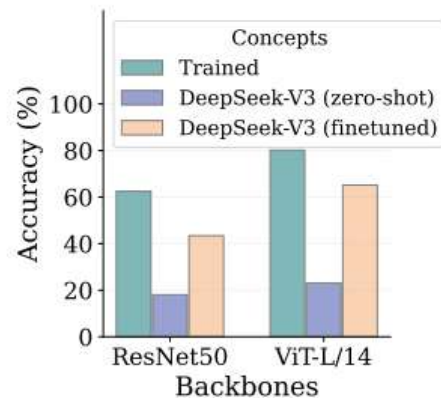




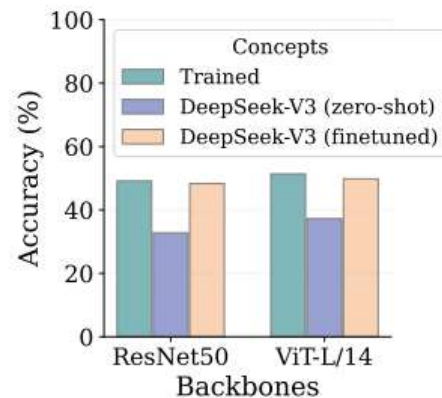
(a) DeepSeek, CIFAR10



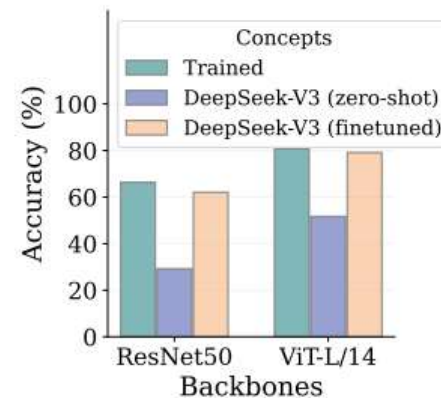
(b) DeepSeek, CIFAR100



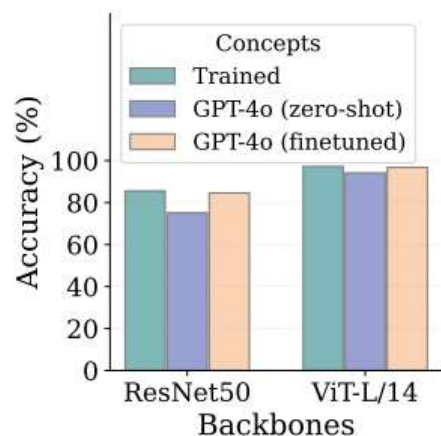
(c) DeepSeek, CUB



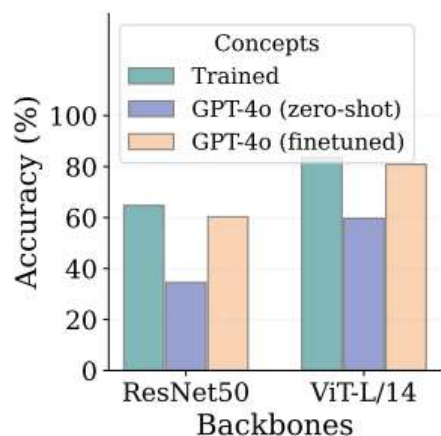
(d) DeepSeek, Places365



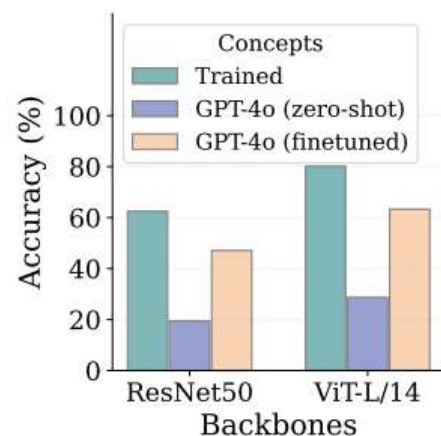
(e) DeepSeek, ImageNet



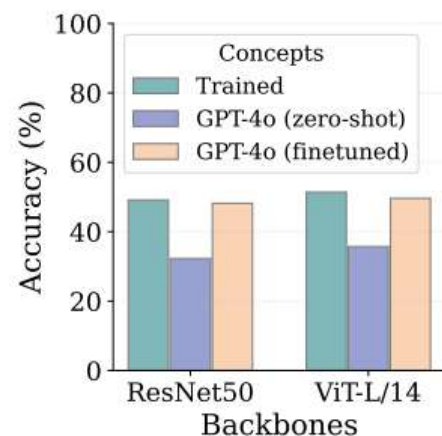
(f) GPT, CIFAR10



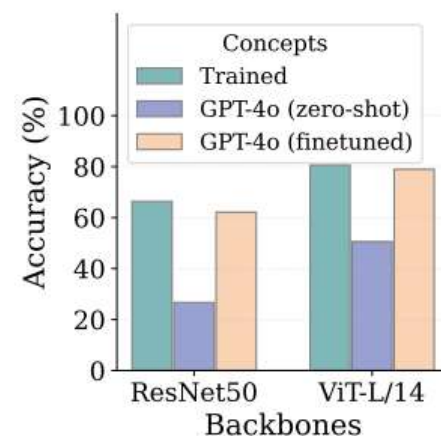
(g) GPT, CIFAR100



(h) GPT, CUB



(i) GPT, Places365



(j) GPT, ImageNet